

PreFlow

Prepress Workflow System — Full System Overview

WS Display | Production Workflow Systems | September 2026

1. Executive Summary

WS Display operates a fully automated order-to-print pipeline. When a customer places an order and uploads artwork, the system processes files, generates proofs, notifies the customer, collects approval, and routes the job to production — all without manual intervention.

The core of this pipeline today is **Callas PDF Toolbox**: two licensed CLI servers that handle all prepress processing. WS Display is replacing Callas with an internally built Python platform. **PreFlow** is the tool that sits at the center of that transition — designed and maintained by Brian Irick, WS Display's Production Workflow Systems Manager, as the control center for how artwork is processed.

Alongside this, **NetSuite** (the current ERP) is being replaced by a custom ERP currently in development.

2. The Current Production Pipeline

The existing pipeline triggers automatically the moment a customer completes an order:

1	Customer orders online Customer places an order on the WS Display website and uploads artwork, stored in AWS S3 immediately.
2	Artwork sent to Callas CLI The order system triggers the Callas CLI server, which pulls the artwork from S3 and runs the configured recipe for that product type.
3	Callas processes the file The recipe runs: preflight checks, CMYK color conversion, multi-panel imposition, overlay and cutpath application. Two outputs are produced: a low-resolution JPEG proof and a press-ready print PDF.
4	Output stored in AWS S3 Both the proof JPEG and print PDF are written back to designated S3 buckets. All production assets — overlays, cutpaths — are also stored there.
5	NetSuite records updated S3 file links are automatically populated into the NetSuite sales order record, visible to internal staff.
6	Customer notified NetSuite sends an automated email; the customer clicks through to review and approve JPEG proofs on the WS Display website.

7

Order goes to production

Once all items are approved, the order automatically routes to production. Print files are pulled from AWS S3 and sent to the press queues.

Two Callas servers: The customer-facing server handles automated online orders. A second internal server handles jobs where the customer emails artwork — staff upload it to the backend and the same Callas pipeline processes it from there. Both carry separate licenses.

3. What's Being Replaced — and Why

Three systems are being retired. Each introduces cost, dependency, or operational constraints that the new platform eliminates:

Callas Toolbox (desktop)

- Used to configure and manage processing recipes for each product type
- Requires a Callas license; recipe knowledge lives in a proprietary environment
- No version control — changes are hard to track or roll back
- **Replaced by: PreFlow** — Python-based, browser-accessible, fully version-controlled on GitHub

Callas CLI Servers (×2)

- Execute processing recipes on every incoming artwork file
- Two licensed servers: one automated (customer-facing), one for manually submitted jobs
- License cost and hardware dependency for every job processed
- **Replaced by: Python processing pipeline** — runs the same logic (preflight, color, imposition, proof generation) on EC2 without Callas, owned entirely by WS Display

NetSuite ERP

- Holds sales order records; surfaces proof and print file S3 links to internal staff
- Sends customer notifications when proofs are ready
- **Replaced by: Custom ERP (in development)** — will receive S3 links from the Python pipeline and manage internal job visibility and customer notifications

4. PreFlow — What It Is and What It Does

PreFlow is the recipe configuration and validation tool for the Python processing pipeline. Designed and maintained by Brian Irick, it is the direct replacement for Callas Toolbox as the environment used to define how each product type is processed — giving WS Display full in-house control over every stage of the prepress pipeline.

PreFlow is an internal tool. It is not in the automated production path and does not handle live customer jobs. Its role is to build and validate processing configurations, then export them to the server where the automated pipeline applies them to every incoming job of that type.

4.1 Processing Capabilities — Expandable by Design

PreFlow defines what happens to every artwork file that passes through the pipeline. The processing capabilities it can configure are not fixed — new checks, steps, and behaviors can be added to the pipeline as the business requires:

- **Preflight validation** — page size, color mode, bleed, resolution, and any custom check that can be defined
- **Color processing** — CMYK conversion, spot color normalization, white ink remapping, LAB conversion, press profile application
- **Imposition** — multi-panel layout assembly with per-panel positioning, rotation, and output sheet sizing
- **Overlay and cutpath application** — product templates and die-lines applied automatically from S3
- **Proof generation** — low-res JPEG proofs for customer approval alongside press-ready PDFs
- **Barcoding and job stamping** — ability to embed barcodes, sales order numbers, or other identifiers into processed files for downstream tracking
- **Custom processing steps** — any new preflight check, fixup, or output format can be added to the pipeline as needs evolve

4.2 How PreFlow Fits into the Workflow

Brian Irick configures a recipe in PreFlow and validates that it produces the correct output for a given product type — correct panel layout, color, cutpath alignment, and proof appearance. Once the recipe is confirmed, it is exported to the production server. From that point, the Python pipeline runs it automatically on every incoming job. PreFlow's involvement for that product type is complete.

Configure	Define every processing step for a product type in PreFlow — preflight rules, color, imposition layout, overlays, cut
Validate	Run a reference file through the pipeline in PreFlow to confirm all steps execute correctly and output matches spe
Export	Push the validated recipe to the production server — the Python pipeline picks it up and applies it to all future jobs
Monitor	Review proof and press file output as needed to verify ongoing pipeline quality — update and re-export recipes wh

4.3 Application Structure

Run Profile	Select a product recipe and run the full pipeline on a reference file. Validates that the recipe executes correctly and generates the expected proof and press file output.
Build Profile	Configure product recipes — overlay, cutpath, impose cutpath, preflight rules, and imposition layout. The impose settings editor allows panel positions to be tuned precisely. Changes export as JSON and push to GitHub for version control.
Preflight Check	Standalone artwork validation — checks size and color mode against product specs without running the full impose pipeline.

5. Technology Stack

Component	Technology	Purpose
Frontend / UI	Streamlit (Python)	Browser-based — no install required
PDF Engine	pypdf + PyMuPDF (fitz)	PDF manipulation, compositing, color handling, rendering
Storage	AWS S3	Overlays, cutpaths, impose cutpaths, output files
Configuration	JSON profiles on GitHub	All product definitions — version-controlled and diff-able
Source Control	GitHub (Brianirick/callas-app)	Version control; auto-deploys on push to main branch
Hosting (current)	Streamlit Cloud	Current deployment — migrating to WS Display EC2
Server pipeline	Python (replacing Callas CLI)	Same processing logic as PreFlow, running automatically on EC2

6. EC2 Deployment Plan

PreFlow deploys to WS Display's existing AWS EC2 infrastructure with no code changes. This is the first step toward a broader migration: the same EC2 environment will ultimately host the Python processing pipeline that replaces both Callas CLI servers.

Server requirements:

- Ubuntu 22.04 LTS or Amazon Linux 2
- t3.small minimum (t3.medium recommended for larger files) — 2 vCPU, 2 GB RAM
- 20 GB EBS storage (expandable)
- IAM role attached with S3 read/write access — no hardcoded credentials
- Port 8501 open inbound (or 80/443 with Nginx in front)

Deployment steps:

Clone repo

```
git clone https://github.com/Brianirick/callas-app.git
```

Install packages

```
pip install -r requirements.txt
```

IAM role

```
Attach role to EC2 – S3 auth is automatic, no keys in code
```

systemd service

```
ExecStart=streamlit run ws_app.py --server.port 8501 (Restart=always)
```

Enable + start

```
sudo systemctl enable preflow && sudo systemctl start preflow
```

Nginx (optional)

```
Reverse proxy 443 → 8501 for HTTPS and a clean internal URL
```

Auto-deploy

GitHub webhook: `git pull && sudo systemctl restart preflow` on push to main

■ **Estimated setup time: 2–4 hours** for a developer familiar with EC2 and Linux. Once PreFlow is running on EC2, the same environment is the natural home for the Python processing pipeline — making the Callas CLI server migration a direct next step.

7. Roadmap

Immediate — Complete PreFlow Profiles

- Finish processing profiles for all current product types
- Create and upload impose cutpath PDFs for all multi-panel products
- Complete position tuning for all product variants

EC2 Migration — PreFlow on EC2

- Deploy PreFlow to WS Display EC2 server
- Configure IAM role for S3 — remove Streamlit Cloud dependency
- Set up Nginx with HTTPS for secure internal access
- Configure auto-deploy from GitHub main branch

Python Pipeline — Replace Callas CLI

- Build automated Python processing server on EC2 using PreFlow's engine
- Route online orders to Python pipeline instead of Callas CLI servers
- Handle both automated (online orders) and manual (emailed artwork) use cases
- Decommission both Callas CLI servers and associated licenses

ERP Integration — Connect to New Custom ERP

- Python pipeline pushes processed file S3 links to new ERP on job completion
- New ERP handles customer proof notifications and internal job visibility
- NetSuite decommissioned once ERP integration is stable

Future — Platform Expansion

- Expand product catalog: additional display types and product categories
- Barcoding and sales order stamping on processed files for production tracking
- Job logging, processing audit trail, and error alerting
- Additional custom preflight checks and processing steps as business needs evolve
- Direct integration between order management and production queues